

Inteligencia Artificial y Mini-Robots

Taller 2

Presentado por:

Cristian Camilo García Gamboa - crigarciaga@unal.edu.co

Daniel Alejandro Torres Sanabria - datorressa@unal.edu.co

Santiago Ospina Contreras - saospinaco@unal.edu.co

Profesor:

José J. Martínez P.

jmcursosingenieria111@gmail.com

Fecha de entrega:

10 de Septiembre de 2026



Universidad Nacional de Colombia - Sede Bogotá
Facultad de Ingeniería
2026 II

1. Observe sus comportamientos en la casa, en la universidad y en el medio de transporte que utiliza. Encuentre, para cada uno de estos escenarios sus reglas básicas.

A. En la casa

Célula: Persona que habita la casa.

Vecindad: Familiares con quienes interactúa diariamente.

Estados posibles:

- Dormido
- Despierto
- Comiendo
- Trabajando/estudiando
- Descansando

Reglas básicas:

1. Si es hora de dormir y la mayoría de los miembros de la familia están descansando, la persona pasa al estado *Dormido*.
2. Si es hora de comer y los demás miembros están reunidos en el comedor, la persona pasa al estado *Comiendo*.
3. Si es día laboral o académico y está despierto, pasa al estado *Trabajando/Estudiando*.
4. Si termina sus actividades y la mayoría de la familia está relajada, pasa al estado *Descansando*.

B. En la universidad

Célula: Estudiante.

Vecindad: Compañeros de clase y docentes cercanos.

Estados posibles:

- En clase
- Participando
- Estudiando
- Descansando
- Ausente

Reglas básicas:

1. Si inicia una clase y el estudiante está presente, pasa al estado *En clase*.
2. Si el profesor realiza preguntas y varios compañeros participan, aumenta la probabilidad de pasar a *Participando*.
3. Si se aproxima un examen o entrega, pasa al estado *Estudiando*.

4. Si finaliza una clase y no tiene actividades inmediatas, pasa a *Descansando*.
5. Si no asiste a la universidad, permanece en estado *Ausente*.

C. En el medio de transporte

Célula: Usuario del transporte.

Vecindad: Pasajeros cercanos y conductor.

Estados posibles:

- Esperando
- Abordando
- Viajando
- Descendiendo

Reglas básicas:

1. Si llega el vehículo y hay espacio disponible, pasa de *Esperando* a *Abordando*.
2. Una vez dentro del vehículo, pasa al estado *Viajando*.
3. Si se aproxima a su destino, cambia a *Descendiendo*.
4. Si el vehículo está lleno, permanece en estado *Esperando*.
5. Si otros pasajeros se desplazan hacia la salida, aumenta la probabilidad de que el usuario también se prepare para descender.

2. Suponga una enfermedad, o un incendio forestal, o una moda, desarrolle un modelo de difusión usando ACs probabilísticos. O simule un robot con dos ruedas que evite obstáculos.

Enfermedad: Gripe Contagiosa

1. Retículo

- Retículo cuadrado de 100×100 celdas.
- Total: 10 000 celdas.

2. Vecindad

- Vecindad de Moore (8 vecinos alrededor de cada célula).

3. Estados (alfabeto)

$w = \{S, I, R, M, V\}$ $w = \{S, I, R, M, V\}$

Donde:

- **S:** Persona sana.
- **I:** Persona infectada.

- **R:** Persona recuperada.
- **M:** Persona fallecida.
- **V:** Celda vacía.

4. Configuración inicial

- 7 900 personas sanas (S).
- 100 personas infectadas (I).
- 2 000 celdas vacías (V).

Distribuidas aleatoriamente.

5. Reglas probabilísticas

Regla 1. Contagio

Si una persona sana tiene nn vecinos infectados:

Vecinos infectados	Probabilidad de contagio
1	15 %
2	30 %
3 o más	60 %

Regla 2. Recuperación

Después de 10 ciclos:

- 85 % de probabilidad de pasar a **Recuperado (R)**.

Regla 3. Fallecimiento

Después de 10 ciclos:

- 15 % de probabilidad de pasar a **Fallecido (M)**.

Regla 4. Inmunidad

Los individuos recuperados permanecen en estado **R** y no vuelven a contagiarse.

Regla 5. Movimiento

En cada ciclo:

- Cada persona puede moverse a una celda vecina libre (Norte, Sur, Este u Oeste) con probabilidad 0,25.
- Si la celda destino está ocupada, permanece en su posición.

CÓDIGO

```
import numpy as np
import random
import matplotlib.pyplot as plt

# PARÁMETROS DEL MODELO

FILAS = 100
COLUMNAS = 100
CICLOS = 100

# ESTADOS

VACIO = 0
SANO = 1
INFECTADO = 2
RECUPERADO = 3
MUERTO = 4

# MATRIZ PRINCIPAL

mundo = np.zeros((FILAS, COLUMNAS), dtype=int)

# CONTADOR DE DÍAS INFECTADO

dias_infectado = np.zeros((FILAS, COLUMNAS), dtype=int)

# CONFIGURACIÓN INICIAL

celdas = [(i, j) for i in range(FILAS) for j in range(COLUMNAS)]
random.shuffle(celdas)

# ASIGNAR PERSONAS SANAS

for i in range(7900):
    x, y = celdas[i]
    mundo[x, y] = SANO

# ASIGNAR PERSONAS INFECTADAS
```

```

for i in range(7900, 8000):
    x, y = celdas[i]
    mundo[x, y] = INFECTADO

# FUNCIÓN VECINDAD DE MOORE

def vecinos_moore(x, y):

    vecinos = []

    for dx in [-1, 0, 1]:
        for dy in [-1, 0, 1]:

            if dx == 0 and dy == 0:
                continue

            nx = (x + dx) % FILAS
            ny = (y + dy) % COLUMNAS

            vecinos.append((nx, ny))

    return vecinos

# FUNCIÓN DE MOVIMIENTO

def mover_personas(matriz):

    nueva = matriz.copy()

    for x in range(FILAS):
        for y in range(COLUMNAS):

            if matriz[x, y] in [SANO, INFECTADO, RECUPERADO]:

                if random.random() < 0.25:

                    movimientos = [
                        (-1, 0),
                        (1, 0),
                        (0, -1),
                        (0, 1)
                    ]

```

```

    dx, dy = random.choice(movimientos)

    nx = (x + dx) % FILAS
    ny = (y + dy) % COLUMNAS

    if nueva[nx, ny] == VACIO:

        nueva[nx, ny] = nueva[x, y]
        nueva[x, y] = VACIO

    return nueva

# HISTORIAL DE ESTADÍSTICAS

hist_sanos = []
hist_infectados = []
hist_recuperados = []
hist_muertos = []

# INICIO DE LA SIMULACIÓN

for ciclo in range(CICLOS):

    nuevo_mundo = mundo.copy()

    # RECORRER TODAS LAS CELDAS

    for x in range(FILAS):
        for y in range(COLUMNAS):

            estado = mundo[x, y]

            # PROCESO DE CONTAGIO

            if estado == SANO:

                vecinos = vecinos_moore(x, y)

                infectados = 0

            # CONTAR VECINOS INFECTADOS

            for vx, vy in vecinos:

```

```

    if mundo[vx, vy] == INFECTADO:
        infectados += 1

# ASIGNAR PROBABILIDAD DE CONTAGIO

if infectados == 1:
    prob = 0.15

elif infectados == 2:
    prob = 0.30

elif infectados >= 3:
    prob = 0.60

else:
    prob = 0

# REALIZAR CONTAGIO

if random.random() < prob:
    nuevo_mundo[x, y] = INFECTADO

# EVOLUCIÓN DE LA ENFERMEDAD

elif estado == INFECTADO:

    dias_infectado[x, y] += 1

# RECUPERACIÓN O MUERTE

if dias_infectado[x, y] >= 10:

    if random.random() < 0.85:
        nuevo_mundo[x, y] = RECUPERADO

    else:
        nuevo_mundo[x, y] = MUERTO

# ACTUALIZAR MUNDO

mundo = nuevo_mundo

# MOVIMIENTO DE PERSONAS

```

```
mundo = mover_personas(mundo)
```

```
# CÁLCULO DE ESTADÍSTICAS
```

```
sanos = np.sum(mundo == SANO)
infectados = np.sum(mundo == INFECTADO)
recuperados = np.sum(mundo == RECUPERADO)
muertos = np.sum(mundo == MUERTO)
```

```
# ALMACENAR HISTORIAL
```

```
hist_sanos.append(sanos)
hist_infectados.append(infectados)
hist_recuperados.append(recuperados)
hist_muertos.append(muertos)
```

```
# MOSTRAR RESULTADOS DEL CICLO
```

```
print(
    f'Ciclo {ciclo:3d} | '
    f'Sanos={sanos:4d} | '
    f'Infectados={infectados:4d} | '
    f'Recuperados={recuperados:4d} | '
    f'Muertos={muertos:4d}'
)
```

```
# GRAFICAR RESULTADOS
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(hist_sanos, label="Sanos")
plt.plot(hist_infectados, label="Infectados")
plt.plot(hist_recuperados, label="Recuperados")
plt.plot(hist_muertos, label="Muertos")
```

```
# CONFIGURACIÓN DE LA GRÁFICA
```

```
plt.xlabel("Ciclos")
plt.ylabel("Cantidad de personas")
plt.title("Difusión de una enfermedad mediante AC probabilístico")
plt.legend()
plt.grid(True)
```

```
# MOSTRAR GRÁFICA
```

plt.show()

3. Simule el comportamiento de un robot con tres sensores de distancia, que recorre un espacio bidimensional, donde hay 4 objetos distribuidos aleatoriamente, que no se choca con esos objetos.

Tomando como referencia los modelos 2D analizados en NetLogo realizamos este código para realizar la simulación del robot como se muestra en el Anexo 1: código robot de 3 sensores; esto realizado en el netlogo code. También, con la opción de Mode, al cambiarla a Authoring realizamos la interfaz para los botones y un slider para la cantidad de objetos, y otra para la velocidad del robot, quedando así:

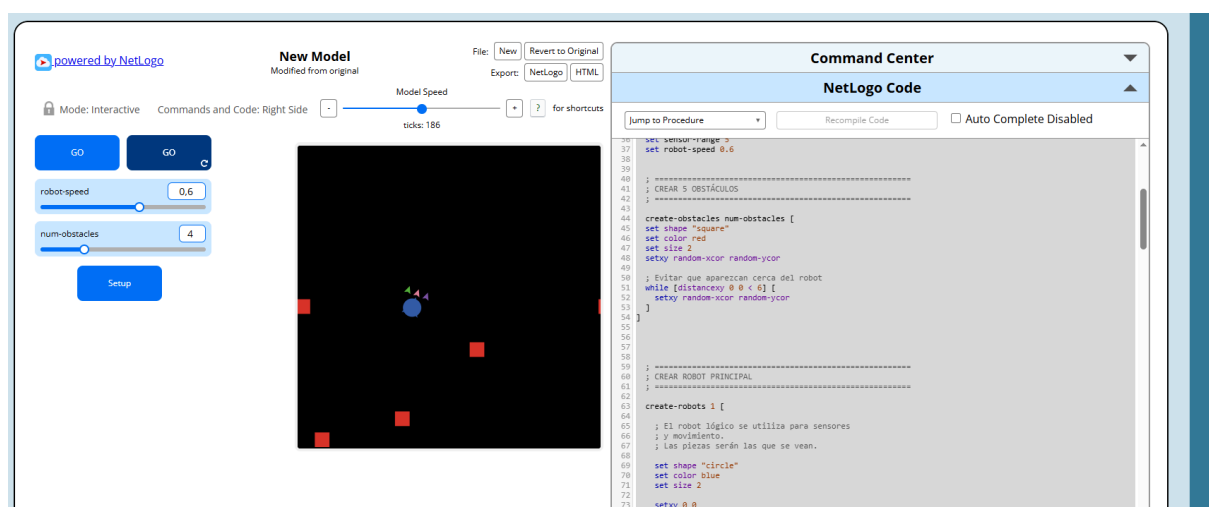


Figura 1: interfaz de Netlogo con código

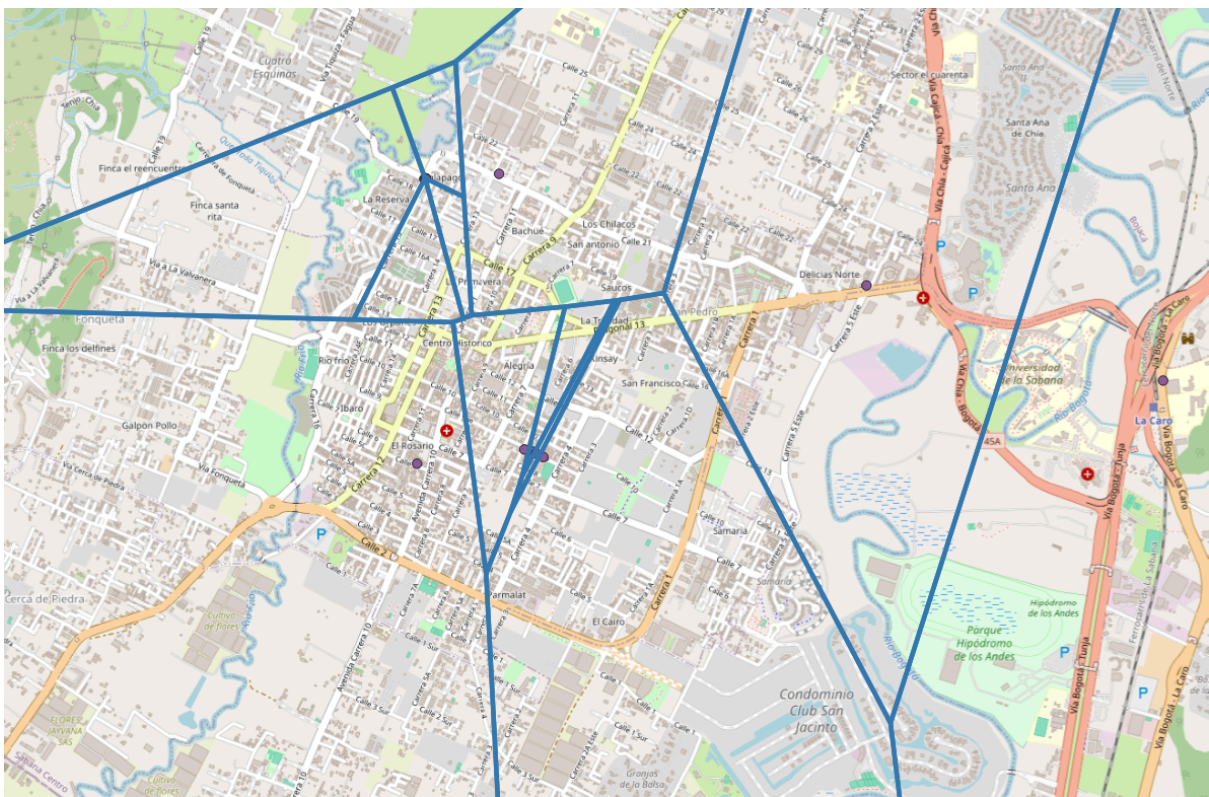
Junto a este documento se enviará también el HTML para probar la simulación en NetLogo.

4. Tome el plano de una ciudad pequeña y localice, por ejemplo, las droguerías, centros de atención de salud y colegios. Por cada concepto dibuje un diagrama de Voronoi. ¿Considera que puede faltar una droguería, o un centro de atención de salud o un colegio? ¿Hay alguna relación entre los diagramas?

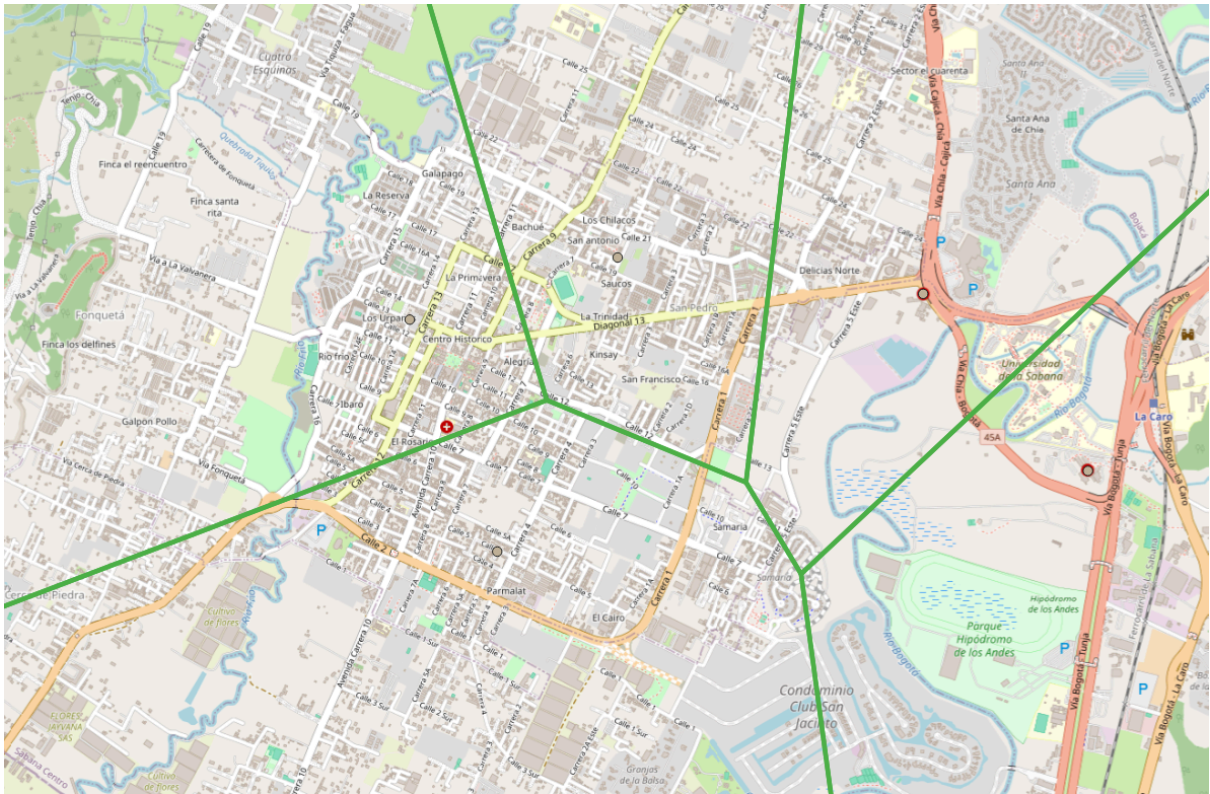
Primero tomaremos como ejemplo la ciudad de Chía (Cundinamarca), situada en la sabana de Bogotá, con una gran dispersión de su población y un crecimiento poblacional en los últimos años, para ayudarnos usaremos el software QGIS que nos ayuda a extraer información clave como los puntos de interés y nos ayuda con la creación de los polígonos. A continuación vamos a ver cada uno de los mapas con los polígonos de Voronoi y su correspondiente búsqueda. Primero vamos a ver el mapa el cual muestra las calles principales y carreras, además de un poco de sus fronteras con otras ciudades.



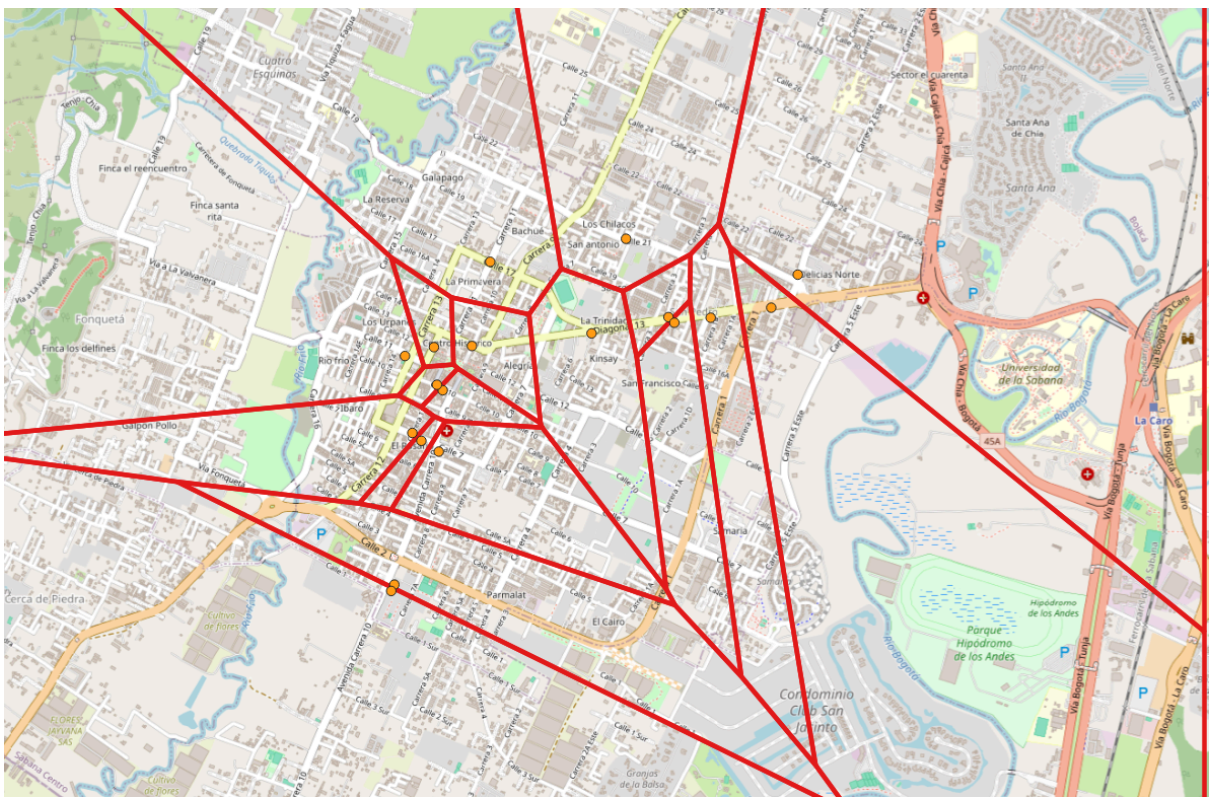
Ahora incluiremos cada uno de los poligonos de voronoi. Vemos el mapa de las escuelas:



Ahora el de los hospitales:



Y por ultimo tenemos el de las droguerías:



Analizando las imágenes vemos que los tres diagramas evidencian vacíos de cobertura en zonas específicas de la ciudad debido a la desproporción geométrica de sus celdas:

- **Centros de Atención de Salud (Líneas Verdes):** Existe una **falta crítica de infraestructura**. El diagrama solo muestra 2 o 3 centros mapeados en toda la ciudad. Esto genera polígonos extremadamente extensos en el oriente (sector cerca a la *Universidad de la Sabana / Parque Hipódromo*) y en el occidente (sector *Fonquetá*). Los residentes de estas periferias deben desplazarse distancias excesivas para llegar al punto más cercano.
- **Colegios / Escuelas (Líneas Azules):** Hay una oferta media-baja. Se observa una concentración de colegios en la franja central y noroccidental. Sin embargo, en el **costado oriental y sur-oriental** las celdas se extienden de forma desmedida, indicando una falta de equipamiento educativo para esa población.
- **Droguerías / Farmacias (Líneas Rojas):** Es el servicio mejor distribuido y más denso. El área urbana central está subdividida en múltiples celdas pequeñas, garantizando alta accesibilidad caminable. Sin embargo, en el **extremo occidente** (zonas rurales/periféricas como *Finca los delfines*) las celdas se alargan de forma drástica, señalando que la periferia carece de droguerías cercanas.

Por otra parte los tres diagramas presentan una clara relación jerárquica y de escala espacial, donde las celdas de droguerías (rojas) constituyen la mallas más granular y densa, quedando múltiples de ellas anidadas dentro de las extensas celdas de los colegios (azules) y hospitales (verdes). Asimismo, existe un patrón de centralidad compartido, ya que las tres mallas concentran sus polígonos más pequeños en el Centro Histórico de la ciudad y se expanden de forma drástica hacia las periferias. Finalmente, se observa una fuerte coincidencia funcional, pues la alta densidad de celdas rojas tiende a aglomerarse justamente en torno a las ubicaciones de los centros de salud y los principales ejes viales, reflejando cómo la infraestructura de salud menor busca la cercanía a los nodos de equipamiento mayor y mayor flujo peatonal.

ANEXOS:

código robot de 3 sensores:

```
globals [
  sensor-range
  robot-speed
  num-obstacles
]
```

```
breed [robots robot]
breed [parts part]
breed [obstacles obstacle]
```

```
robots-own [
  sensor-left
  sensor-front
```

```

    sensor-right
]

parts-own [
  part-type
  offset-x
  offset-y
  parent-robot
]

to setup
  clear-all

  set sensor-range 5
  set robot-speed 0.6

  create-obstacles num-obstacles [
    set shape "square"
    set color red
    set size 2
    setxy random-xcor random-ycor

    while [distancexy 0 0 < 6] [
      setxy random-xcor random-ycor
    ]
  ]

  create-robots 1 [
    set shape "circle"
    set color blue
    set size 2
    setxy 0 0
    set heading random 360
  ]

  ask robots [
    let my-robot self
    hatch-parts 1 [
      set part-type "body"
      set offset-x 0
      set offset-y 0
      set parent-robot my-robot
      set shape "robot-body"
      set color blue
      set size 2.5
    ]
  ]

```

```
hatch-parts 1 [  
  set part-type "wheel-left"  
  set offset-x -1.6  
  set offset-y 0  
  set parent-robot my-robot  
  set shape "robot-wheel"  
  set color black  
  set size 1.5  
]
```

```
hatch-parts 1 [  
  set part-type "wheel-right"  
  set offset-x 1.6  
  set offset-y 0  
  set parent-robot my-robot  
  set shape "robot-wheel"  
  set color black  
  set size 1.5  
]
```

```
hatch-parts 1 [  
  set part-type "front"  
  set offset-x 0  
  set offset-y 1.65  
  set parent-robot my-robot  
  set shape "robot-front"  
  set color pink  
  set size 1.0  
]
```

```
hatch-parts 1 [  
  set part-type "triangle-left"  
  set offset-x -1.05  
  set offset-y 1.65  
  set parent-robot my-robot  
  set shape "robot-triangle"  
  set color green  
  set size 1.0  
]
```

```
hatch-parts 1 [  
  set part-type "triangle-right"  
  set offset-x 1.05  
  set offset-y 1.65  
  set parent-robot my-robot
```

```

    set shape "robot-triangle"
    set color violet
    set size 1.0
  ]
]

ask robots [
  hide-turtle
  update-robot-parts
]

reset-ticks
end

to go
  ask robots [
    read-sensors
    avoid-obstacles
    move-robot
    update-robot-parts
  ]
  tick
end

to read-sensors
  set sensor-left sensor-distance -45
  set sensor-front sensor-distance 0
  set sensor-right sensor-distance 45
end

to-report sensor-distance [relative-angle]
  let center nobody

  if relative-angle = 0 [
    set center patch-ahead sensor-range
  ]
  if relative-angle = -45 [
    set center patch-left-and-ahead 45 sensor-range
  ]

  if relative-angle = 45 [
    set center patch-right-and-ahead 45 sensor-range
  ]

  if center = nobody [
    report sensor-range + 1
  ]
]

```

```

let cx [pxcor] of center
let cy [pycor] of center
let sensor-patches patches with [
  abs (pxcor - cx) <= 2 and
  abs (pycor - cy) <= 2
]

let detected-obstacles obstacles with [
  member? patch-here sensor-patches
]

ifelse any? detected-obstacles [
  report min [
    distancexy
    [xcor] of myself
    [ycor] of myself
  ] of detected-obstacles
] [
  report sensor-range + 1
]
end

to avoid-obstacles
  if sensor-front <= sensor-range [
    if sensor-left > sensor-right [
      left 25
    ]

    if sensor-right > sensor-left [
      right 25
    ]

    if sensor-left = sensor-right [
      ifelse random 2 = 0 [
        left 45
      ] [
        right 45
      ]
    ]
  ]

  if sensor-left <= 2 [
    right 20
  ]

```

```

if sensor-right <= 2 [
  left 20
]
end

to move-robot

if xcor >= max-pxcor - 1 [
  set heading 315
]

if xcor <= min-pxcor + 1 [
  set heading 135
]

if ycor >= max-pycor - 1 [
  set heading 225
]

if ycor <= min-pycor + 1 [
  set heading 45
]

if can-move? robot-speed [
  forward robot-speed
]

update-robot-parts

end

to update-robot-parts

let my-robot self
let r-heading [heading] of my-robot
let r-x [xcor] of my-robot
let r-y [ycor] of my-robot

ask parts with [parent-robot = my-robot] [
  let local-x offset-x
  let local-y offset-y
  let new-x r-x
    + local-x * cos r-heading
    + local-y * sin r-heading

  let new-y r-y
    - local-x * sin r-heading
    + local-y * cos r-heading

```

```
    setxy new-x new-y
    set heading r-headin
  ]
end
```